```graphql
query GetPatients {
  patients {
    id
    name
  }
}
```

Variables Headers

```
1
```

GraphiQL


```
{
  "data": {
    "patients": [
      {
        "id": "1",
        "name": "John Doe"
      },
      {
        "id": "2",
        "name": "Jane Smith"
      },
      {
        "id": "3",
        "name": "Robert Wilson"
      },
      {
        "id": "58396379",
        "name": "Adarsh"
      }
    ]
  }
}
```


```graphql
query {
  tests {
    id
    name
    price
  }
}
```

Variables Headers

```
1
```

GraphiQL


```
{
  "data": {
    "tests": [
      {
        "id": "1",
        "name": "Complete Blood Count",
        "price": 50
      },
      {
        "id": "2",
        "name": "Blood Chemistry Panel",
        "price": 75
      },
      {
        "id": "3",
        "name": "Urinalysis",
        "price": 30
      },
      {
        "id": "4",
        "name": "Thyroid Function Test",
        "price": 60
      }
    ]
  }
}
```

```graphql
query GetPatientSamples {
  patient(id: "1") {
    id
    name
    samples {
      id
      status
      collectedAt
      test {
        name
      }
      result {
        value
        referenceRange
        status
      }
    }
  }
}
```

Variables Headers

```
1
```

GraphiQL


```
{
  "data": {
    "patient": {
      "id": "1",
      "name": "John Doe",
      "samples": [
        {
          "id": "1",
          "status": "collected",
          "collectedAt": "2026-02-25T09:30:00Z",
          "test": {
            "name": "Complete Blood Count"
          },
          "result": {
            "value": "150",
            "referenceRange": "140-180",
            "status": "abnormal"
          }
        }
      ]
    }
  }
}
```


```graphql
query GetTests {
  tests {
    id
    name
    price
    turnaroundTime
    category
  }
}
```

Variables Headers

```
1
```

GraphiQL


```
{
  "data": {
    "tests": [
      {
        "id": "1",
        "name": "Complete Blood Count",
        "price": 50,
        "turnaroundTime": "24 hours",
        "category": "blood"
      },
      {
        "id": "2",
        "name": "Blood Chemistry Panel",
        "price": 75,
        "turnaroundTime": "24 hours",
        "category": "blood"
      },
      {
        "id": "3",
        "name": "Urinalysis",
        "price": 30,
        "turnaroundTime": "4 hours",
        "category": "urine"
      },
      {
        "id": "4",
        "name": "Thyroid Function Test",
        "price": 60,
        "turnaroundTime": "48 hours",
        "category": "blood"
      }
    ]
  }
```

```graphql
query GetSamples {
  samples {
    id
    patientId
    testId
    status
    collectedAt
  }
}
```

Execute query (Ctrl-Enter)

Variables Headers

```
1
```

GraphiQL

```
{
  "samples": [
    {
      "id": "1",
      "patientId": "1",
      "testId": "1",
      "status": "collected",
      "collectedAt": "2026-02-25T09:30:00Z"
    },
    {
      "id": "2",
      "patientId": "2",
      "testId": "2",
      "status": "in_progress",
      "collectedAt": "2026-02-26T10:00:00Z"
    },
    {
      "id": "3",
      "patientId": "3",
      "testId": "3",
      "status": "completed",
      "collectedAt": "2026-02-24T14:15:00Z"
    }
  ]
}
}
```

```graphql
query GetPatient {
  patient(id: "1") {
    id
    name
    samples {
      id
      status
      collectedAt
      test {
        name
      }
      result {
        value
        referenceRange
        status
      }
    }
  }
}
```

Variables Headers

```
1
```

GraphiQL

```
{
  "data": {
    "patient": {
      "id": "1",
      "name": "John Doe",
      "samples": [
        {
          "id": "1",
          "status": "collected",
          "collectedAt": "2026-02-25T09:30:00Z",
          "test": {
            "name": "Complete Blood Count"
          },
          "result": {
            "value": "150",
            "referenceRange": "140-180",
            "status": "abnormal"
          }
        }
      ]
    }
  }
}
```

```python
import strawberry
from fastapi import FastAPI
from strawberry.fastapi import GraphQLRouter
from typing import List, Optional
from datetime import datetime
import motor.motor_asyncio
import uuid
import os
```

```python
MONGO_URL = os.getenv("MONGO_URL", "mongodb://localhost:27017/")
client = motor.motor_asyncio.AsyncIOMotorClient(MONGO_URL)
db = client["medical_lab"]

def get_id():
    return str(uuid.uuid4())[:8]

@strawberry.type
class Lab:
    id: str
    name: str
    address: str
    accreditation: str

@strawberry.type
class Technician:
    id: str
    name: str
    certification: str
    specialization: str

@strawberry.type
class Test:
    id: str
    name: str
    category: str
    price: float
    turnaround_time: str
    unit: str
    lab_id: str

    @strawberry.field
    async def lab(self) -> Optional[Lab]:
        doc = await db.labs.find_one({"id": self.lab_id}, {"_id": 0})
        return Lab(**doc) if doc else None

@strawberry.type
class Result:
    id: str
    sample_id: str
    value: str
    reference_range: str
    status: str

@strawberry.type
```

```python
class Sample:
    id: str
    patient_id: str
    test_id: str
    collected_at: str
    status: str
    technician_id: Optional[str] = None
    test_ids: List[str] = strawberry.field(default_factory=list)

    @strawberry.field
    async def test(self) -> Optional[Test]:
        doc = await db.tests.find_one({"id": self.test_id}, {"_id": 0})
        return Test(**doc) if doc else None

    @strawberry.field
    async def tests(self) -> List[Test]:
        docs = await db.tests.find({"id": {"$in": self.test_ids}}, {"_id":
0}).to_list(length=100)
        return [Test(**doc) for doc in docs]

    @strawberry.field
    async def result(self) -> Optional[Result]:
        doc = await db.results.find_one({"sample_id": self.id}, {"_id": 0})
        return Result(**doc) if doc else None

    @strawberry.field
    async def technician(self) -> Optional[Technician]:
        if not self.technician_id:
            return None
        doc = await db.technicians.find_one({"id": self.technician_id}, {"_id": 0})
        return Technician(**doc) if doc else None

@strawberry.type
class Patient:
    id: str
    name: str
    date_of_birth: str
    gender: str
    email: str
    phone: str

    @strawberry.field
    async def samples(self) -> List[Sample]:
        docs = await db.samples.find({"patient_id": self.id}, {"_id":
0}).to_list(length=100)
```

```
        return [Sample(**doc) for doc in docs]

@strawberry.type
class Query:
    @strawberry.field
    async def patient(self, id: str) -> Optional[Patient]:
        doc = await db.patients.find_one({"id": id}, {"_id": 0})
        return Patient(**doc) if doc else None

    @strawberry.field
    async def patients(self) -> List[Patient]:
        docs = await db.patients.find({}, {"_id": 0}).to_list(length=1000)
        return [Patient(**doc) for doc in docs]

    @strawberry.field
    async def sample(self, id: str) -> Optional[Sample]:
        doc = await db.samples.find_one({"id": id}, {"_id": 0})
        return Sample(**doc) if doc else None

    @strawberry.field
    async def samples(self) -> List[Sample]:
        docs = await db.samples.find({}, {"_id": 0}).to_list(length=1000)
        return [Sample(**doc) for doc in docs]

    @strawberry.field
    async def tests(self, category: Optional[str] = None) -> List[Test]:
        query = {"category": category} if category else {}
        docs = await db.tests.find(query, {"_id": 0}).to_list(length=1000)
        return [Test(**doc) for doc in docs]

    @strawberry.field
    async def results(self) -> List[Result]:
        docs = await db.results.find({}, {"_id": 0}).to_list(length=1000)
        return [Result(**doc) for doc in docs]

@strawberry.type
class Mutation:
    @strawberry.mutation
    async def create_patient(self, name: str, dob: str, gender: str, email: str,
phone: str) -> Patient:
        doc = {
            "id": get_id(), "name": name, "date_of_birth": dob,
            "gender": gender, "email": email, "phone": phone
        }
        await db.patients.insert_one(doc.copy())
```

```
        return Patient(**doc)

    @strawberry.mutation
    async def create_sample(self, patient_id: str, test_id: str, test_ids:
Optional[List[str]] = None) -> Sample:
        tests = test_ids if test_ids else [test_id]
        doc = {
            "id": get_id(), "patient_id": patient_id, "test_id": test_id,
            "test_ids": tests, "collected_at": datetime.now().isoformat(),
            "status": "collected", "technician_id": None
        }
        await db.samples.insert_one(doc.copy())
        return Sample(**doc)

    @strawberry.mutation
    async def record_result(self, sample_id: str, value: str, reference_range: str)
-> Result:
        status = "normal"
        try:
            val = float(value)
            min_v, max_v = map(float, reference_range.split('-'))
            if val < min_v or val > max_v:
                status = "abnormal"
        except ValueError:
            pass

        doc = {
            "id": get_id(), "sample_id": sample_id, "value": value,
            "reference_range": reference_range, "status": status
        }
        await db.results.insert_one(doc.copy())
        await db.samples.update_one({"id": sample_id}, {"$set": {"status":
"completed"}})
        return Result(**doc)

    @strawberry.mutation
    async def create_test(self, name: str, category: str, price: float,
turnaround_time: str, unit: str, lab_id: str) -> Test:
        doc = {
            "id": get_id(), "name": name, "category": category,
            "price": price, "turnaround_time": turnaround_time,
            "unit": unit, "lab_id": lab_id
        }
        await db.tests.insert_one(doc.copy())
        return Test(**doc)
```

```
    @strawberry.mutation
    async def create_lab(self, name: str, address: str, accreditation: str) -> Lab:
        doc = {"id": get_id(), "name": name, "address": address, "accreditation": 
accreditation}
        await db.labs.insert_one(doc.copy())
        return Lab(**doc)

    @strawberry.mutation
    async def create_technician(self, name: str, certification: str, specialization: 
str) -> Technician:
        doc = {"id": get_id(), "name": name, "certification": certification, 
"specialization": specialization}
        await db.technicians.insert_one(doc.copy())
        return Technician(**doc)

    @strawberry.mutation
    async def update_sample_status(self, sample_id: str, status: str) -> 
Optional[Sample]:
        await db.samples.update_one({"id": sample_id}, {"$set": {"status": status}})
        doc = await db.samples.find_one({"id": sample_id}, {"_id": 0})
        return Sample(**doc) if doc else None

schema = strawberry.Schema(query=Query, mutation=Mutation)
graphql_app = GraphQLRouter(schema, graphql_ide="graphql-playground")

app = FastAPI(title="Medical Lab API")
app.include_router(graphql_app, prefix="/graphql")
```